

SQL

Beginner to Advanced

Complete Easy-English Study Material

SQL Learning Path
Database → Tables → SQL Commands → Queries → Results
Theory → Examples → Practice → Interview Preparation

Watermark: BhavPath

BhavPath

1. What is SQL?

SQL means Structured Query Language. It is used to communicate with relational databases. We can create tables, insert data, read data, update data, delete data, and manage database objects.

2. Database, DBMS and RDBMS

A database is an organized collection of data. DBMS is software used to manage databases. RDBMS stores related data in tables and supports relationships using keys.

3. SQL Command Types

DDL: CREATE, ALTER, DROP, TRUNCATE. DML: INSERT, UPDATE, DELETE. DQL: SELECT. DCL: GRANT, REVOKE. TCL: COMMIT, ROLLBACK, SAVEPOINT.

4. CREATE DATABASE and TABLE

```
CREATE DATABASE College;  
USE College;  
CREATE TABLE Student (id INT, name VARCHAR(50), age INT, department VARCHAR(30));
```

```
CREATE DATABASE College;  
USE College;  
CREATE TABLE Student (id INT, name VARCHAR(50), age INT, department VARCHAR(30));
```

5. SQL Data Types

INT for whole numbers; DECIMAL for exact decimals; VARCHAR for variable-length text; CHAR for fixed-length text; DATE for dates; TIMESTAMP/DATETIME for date and time depending on the database.

6. Constraints

PRIMARY KEY uniquely identifies a row. FOREIGN KEY connects tables. NOT NULL prevents missing values. UNIQUE prevents duplicates. DEFAULT supplies a default value. CHECK validates a condition.

7. INSERT

```
INSERT INTO Student (id,name,age,department) VALUES (1,'Ravi',20,'CSE');
INSERT INTO Student VALUES (2,'Anu',21,'ECE');
```

```
INSERT INTO Student (id,name,age,department) VALUES (1,'Ravi',20,'CSE');
INSERT INTO Student VALUES (2,'Anu',21,'ECE');
```

8. SELECT

```
SELECT * FROM Student;
SELECT name,department FROM Student;
SELECT DISTINCT department FROM Student;
```

```
SELECT * FROM Student;
SELECT name,department FROM Student;
SELECT DISTINCT department FROM Student;
```

9. WHERE

WHERE filters rows.

```
SELECT * FROM Student WHERE age > 20;
SELECT * FROM Student WHERE department='CSE';
```

```
WHERE filters rows.
SELECT * FROM Student WHERE age > 20;
SELECT * FROM Student WHERE department='CSE';
```

10. AND, OR, NOT

```
SELECT * FROM Student WHERE age>=20 AND department='CSE';
SELECT * FROM Student WHERE department='CSE' OR department='ECE';
```

```
SELECT * FROM Student WHERE age>=20 AND department='CSE';
SELECT * FROM Student WHERE department='CSE' OR department='ECE';
```

11. ORDER BY

ORDER BY sorts results.

```
SELECT * FROM Student ORDER BY age ASC;  
SELECT * FROM Student ORDER BY age DESC;
```

ORDER BY sorts results.

```
SELECT * FROM Student ORDER BY age ASC;  
SELECT * FROM Student ORDER BY age DESC;
```

12. LIKE

LIKE searches patterns. % means any number of characters and _ means one character.

```
SELECT * FROM Student WHERE name LIKE 'A%';
```

LIKE searches patterns. % means any number of characters and _ means one character.

```
SELECT * FROM Student WHERE name LIKE 'A%';
```

13. IN and BETWEEN

```
SELECT * FROM Student WHERE department IN ('CSE','ECE');  
SELECT * FROM Student WHERE age BETWEEN 18 AND 22;
```

```
SELECT * FROM Student WHERE department IN ('CSE','ECE');  
SELECT * FROM Student WHERE age BETWEEN 18 AND 22;
```

14. NULL

NULL means missing or unknown. Use IS NULL and IS NOT NULL.

```
SELECT * FROM Student WHERE email IS NULL;
```

NULL means missing or unknown. Use IS NULL and IS NOT NULL.

```
SELECT * FROM Student WHERE email IS NULL;
```

15. Aggregate Functions

COUNT(), SUM(), AVG(), MIN(), MAX() summarize data.

```
SELECT COUNT(*) FROM Student;  
SELECT AVG(age) FROM Student;
```

COUNT(), SUM(), AVG(), MIN(), MAX() summarize data.

```
SELECT COUNT(*) FROM Student;  
SELECT AVG(age) FROM Student;
```

16. GROUP BY

GROUP BY creates groups for aggregate calculations.

```
SELECT department, COUNT(*) FROM Student GROUP BY department;
```

GROUP BY creates groups for aggregate calculations.

```
SELECT department, COUNT(*) FROM Student GROUP BY department;
```

17. HAVING

HAVING filters groups after grouping.

```
SELECT department, COUNT(*) FROM Student GROUP BY department HAVING COUNT(*) > 2;
```

HAVING filters groups after grouping.

```
SELECT department, COUNT(*) FROM Student GROUP BY department HAVING COUNT(*) > 2;
```

18. UPDATE

UPDATE changes existing rows.

```
UPDATE Student SET age=21 WHERE id=1;
```

Always check the WHERE condition.

UPDATE changes existing rows.

```
UPDATE Student SET age=21 WHERE id=1;
```

Always check the WHERE condition.

19. DELETE

DELETE removes rows.

```
DELETE FROM Student WHERE id=3;
```

Without WHERE, all rows may be removed.

DELETE removes rows.

```
DELETE FROM Student WHERE id=3;
```

Without WHERE, all rows may be removed.

20. TRUNCATE and DROP

TRUNCATE removes table rows while keeping the table structure in systems that support it. DROP removes the database object itself.

21. ALTER TABLE

ALTER changes table structure.

```
ALTER TABLE Student ADD email VARCHAR(100);
```

Exact syntax for rename/drop can vary by database.

ALTER changes table structure.

```
ALTER TABLE Student ADD email VARCHAR(100);
```

Exact syntax for rename/drop can vary by database.

22. Primary Key

A primary key uniquely identifies each row and normally cannot be NULL. Example: student_id.

23. Foreign Key

A foreign key references a key in another table and creates a relationship.

```
FOREIGN KEY(student_id) REFERENCES Student(id)
```

A foreign key references a key in another table and creates a relationship.

```
FOREIGN KEY(student_id) REFERENCES Student(id)
```

24. Relationships

One-to-one: one record matches one. One-to-many: one customer can have many orders. Many-to-many: many students can take many courses, usually through a bridge table.

25. INNER JOIN

Returns matching rows from both tables.

```
SELECT s.name,o.order_id FROM Student s INNER JOIN Orders o ON s.id=o.student_id;
```

Returns matching rows from both tables.

```
SELECT s.name,o.order_id FROM Student s INNER JOIN Orders o ON s.id=o.student_id;
```

26. LEFT JOIN

Returns all rows from the left table and matching rows from the right table.

27. RIGHT and FULL OUTER JOIN

RIGHT JOIN returns all rows from the right table plus matches. FULL OUTER JOIN returns rows from both sides. Support varies by database.

28. SELF JOIN

A table can join to itself, useful for employee-manager relationships.

29. CROSS JOIN

Produces combinations of rows from two tables. Use carefully because results can become very large.

30. Subqueries

A subquery is a query inside another query.

```
SELECT name FROM Student WHERE age > (SELECT AVG(age) FROM Student);
```

A subquery is a query inside another query.

```
SELECT name FROM Student WHERE age > (SELECT AVG(age) FROM Student);
```

BhavPath

31. EXISTS

EXISTS checks whether a subquery returns at least one row.

```
SELECT name FROM Student s WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.student_id=s.id);
```

EXISTS checks whether a subquery returns at least one row.

```
SELECT name FROM Student s WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.student_id=s.id);
```

32. CASE

CASE creates conditional output.

```
SELECT name, CASE WHEN age<18 THEN 'Minor' WHEN age<22 THEN 'Young' ELSE 'Adult' END AS category  
FROM Student;
```

CASE creates conditional output.

```
SELECT name, CASE WHEN age<18 THEN 'Minor' WHEN age<22 THEN 'Young' ELSE 'Adult' END AS category FROM Student;
```

33. Views

A view is a saved query presented like a virtual table.

```
CREATE VIEW CSE_Students AS SELECT id,name FROM Student WHERE department='CSE';
```

A view is a saved query presented like a virtual table.

```
CREATE VIEW CSE_Students AS SELECT id,name FROM Student WHERE department='CSE';
```

34. Indexes

An index can improve search performance for suitable queries.

```
CREATE INDEX idx_name ON Student(name);
```

Indexes use storage and can add write overhead.

An index can improve search performance for suitable queries.

```
CREATE INDEX idx_name ON Student(name);
```

Indexes use storage and can add write overhead.

35. Normalization

Normalization reduces unnecessary duplication. 1NF uses atomic values. 2NF removes partial dependency. 3NF removes inappropriate transitive dependency.

36. Transactions

A transaction is a logical unit of work. COMMIT saves changes; ROLLBACK undoes uncommitted changes; SAVEPOINT creates a rollback point.

37. ACID

Atomicity: all or nothing. Consistency: valid state. Isolation: transactions do not improperly interfere. Durability: committed changes persist.

38. DCL

GRANT gives permissions and REVOKE removes permissions. Exact syntax depends on the database system.

39. Stored Procedures

A stored procedure is reusable SQL logic stored in the database. Syntax varies among MySQL, PostgreSQL, SQL Server, and Oracle.

40. Functions

Databases provide built-in string, numeric, date, and aggregate functions. Some also support user-defined functions.

BhavPath

41. Triggers

A trigger runs automatically after/before selected events such as INSERT, UPDATE, or DELETE. Useful for auditing, but excessive triggers can complicate systems.

42. SQL Execution Order

Simplified logical order: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET.

43. SQL and Data Science

SQL is used to extract, filter, clean, join, and aggregate data before analysis in Python, Power BI, notebooks, or other tools.

44. SQL vs NoSQL

SQL databases use relational tables and structured relationships. NoSQL includes document, key-value, wide-column, and graph databases. The choice depends on the application.

45. SQL Security

Use parameterized queries/prepared statements. Never concatenate untrusted input directly into SQL. Use least-privilege accounts and protect credentials.

BhavPath

46. SQL Quick Cheat Sheet

Task	Pattern
Read	SELECT ... FROM ...;
Filter	WHERE condition
Sort	ORDER BY column ASC/DESC
Group	GROUP BY column
Group filter	HAVING condition
Insert	INSERT INTO ... VALUES ...
Update	UPDATE ... SET ... WHERE ...
Delete	DELETE FROM ... WHERE ...
Join	JOIN ... ON ...
View	CREATE VIEW ... AS SELECT ...
Index	CREATE INDEX ... ON ...

BhavPath

47. SQL Practice Questions

1. Create a Student table.
2. Insert five students.
3. Display all students.
4. Display only names.
5. Find students older than 20.
6. Find CSE students.
7. Sort by age descending.
8. Show unique departments.
9. Count students.
10. Find average age.
11. Count students by department.
12. Use HAVING with COUNT.
13. Find names starting with A.
14. Use BETWEEN.
15. Use IN.
16. Update one student.
17. Delete one student.
18. Create Course and Enrollment tables.
19. Write an INNER JOIN.
20. Write a LEFT JOIN.
21. Find students above average age.
22. Create a view.
23. Create an index.
24. Use CASE to categorize students.
25. Write a transaction.

BhavPath

48. Mini Project: College Database

STUDENT → ENROLLMENT → COURSE. Use student_id and course_id as keys.

Example table design

```
CREATE TABLE Student( student_id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, department VARCHAR(30), age INT );
```

```
CREATE TABLE Course( course_id INT PRIMARY KEY, course_name VARCHAR(80), credits INT );
```

```
CREATE TABLE Enrollment( student_id INT, course_id INT, semester INT, PRIMARY KEY(student_id,course_id), FOREIGN  
KEY(student_id) REFERENCES Student(student_id), FOREIGN KEY(course_id) REFERENCES Course(course_id) );
```

Example report query

```
SELECT s.department, COUNT(*) AS total  
FROM Student s  
JOIN Enrollment e ON s.student_id=e.student_id  
GROUP BY s.department;
```

BhavPath

49. SQL Interview Questions

What is SQL?

A language used to work with relational databases.

What is a primary key?

A unique identifier for table rows.

What is a foreign key?

A column that references a key in another table.

WHERE vs HAVING?

WHERE filters rows; HAVING filters groups.

What is JOIN?

It combines related data from tables.

What is normalization?

A design technique to reduce unnecessary duplication.

What is an index?

A structure that can speed up suitable data retrieval.

What is a view?

A saved query presented as a virtual table.

What is NULL?

A missing or unknown value.

What is a transaction?

A logical unit of database work.

What are ACID properties?

Atomicity, Consistency, Isolation, Durability.

What is a subquery?

A query nested inside another query.

What is SQL injection?

An attack where unsafe input changes the intended SQL query.

How can SQL injection be reduced?

Use parameterized queries/prepared statements and validate input.

Why use GROUP BY?

To create groups for aggregate calculations.

50. Final Revision Checklist

- ✓ Database and RDBMS
- ✓ Tables, rows, columns
- ✓ Data types
- ✓ DDL, DML, DQL, DCL, TCL
- ✓ CREATE, INSERT, SELECT, UPDATE, DELETE
- ✓ WHERE, ORDER BY, GROUP BY, HAVING
- ✓ Aggregate functions
- ✓ Constraints
- ✓ Primary and foreign keys
- ✓ Joins
- ✓ Subqueries and EXISTS
- ✓ CASE
- ✓ Views and indexes
- ✓ Normalization
- ✓ Transactions and ACID
- ✓ SQL security
- ✓ SQL for data science

Keep practicing SQL every day. Start with small tables, write queries, and check the result.

BhavPath • Complete SQL Learning Material

BhavPath